

**IZMIR KATIP CELEBI UNIVERSITY**

FACULTY OF ENGINEERING AND ARCHITECTURE

DEPARTMENT OF COMPUTER ENGINEERING

**CENG 454 - Artificial Intelligence and Data Science**

Final Project Report

---

## **Fake News Detection Using TF-IDF and Transformer-Based Sentence Embeddings**

---

**Group No: 13**

| <b>Student Name &amp; Surname</b> | <b>Student Number</b> |
|-----------------------------------|-----------------------|
| Samet Ünsal                       | 220401082             |
| Fatmanur Bulut                    | 230401034             |
| Eren Makraç                       | 210401035             |

**Submission Date:** May 22, 2026

## Abstract

This project investigates fake news detection as a supervised binary text classification task on a Turkish-language news dataset. The main objective is to compare a traditional lexical representation method, TF-IDF, with transformer-based sentence embeddings for distinguishing between fake and real news. In the baseline approach, TF-IDF vectors were extracted from news texts and classified using Logistic Regression. In the embedding-based approach, dense sentence representations were generated with the multilingual Sentence-BERT model `paraphrase-multilingual-MiniLM-L12-v2`, and two classical classifiers, Logistic Regression and Support Vector Machine (SVM), were trained on these representations. The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices on a fixed stratified test set. The experimental results show that TF-IDF + Logistic Regression achieved the highest overall performance with an accuracy of 0.9381 and an F1-score of 0.9317. Although transformer-based embeddings produced competitive results, the TF-IDF baseline performed better in this experimental setup. This suggests that lexical patterns in the selected dataset were highly informative for fake news classification.

## 1 Introduction and Motivation

The rapid growth of online news platforms and social media has increased the speed at which information spreads. While this enables faster access to news, it also makes the distribution of misleading or fabricated information easier. Fake news can influence public opinion, reduce trust in legitimate media sources, and create social, political, or economic harm. Therefore, automatic fake news detection has become an important research problem in artificial intelligence, data science, and natural language processing.

Fake news detection is commonly formulated as a text classification task. Given a news article, headline, or short description, the goal is to determine whether the content is reliable or misleading. In this project, the task is defined as binary classification with two labels: `fake` and `real`. The project focuses on Turkish news texts, which makes the task especially relevant because many fake news detection studies and benchmark datasets are centered on English. Working with Turkish also introduces additional challenges related to morphology, vocabulary variation, and dataset availability.

Traditional machine learning approaches have been widely used for fake news detection. One of the most common methods is TF-IDF, which represents a document according to the importance of its words and n-grams. TF-IDF is simple, efficient, and often effective when class-specific lexical patterns exist in the dataset. For this reason, it provides a strong baseline for evaluating more advanced representation methods.

Transformer-based language models offer a different approach. Instead of representing text only through word frequency, transformer models use attention mechanisms to learn contextual and semantic relationships between words. Sentence embedding models, such as Sentence-BERT, can convert a sentence or document into a dense numerical vector that represents its semantic content. These embeddings can then be used as input features for machine learning classifiers.

The main motivation of this project is to examine whether transformer-based sentence embeddings improve fake news detection performance compared to a traditional TF-IDF baseline. For this purpose, the project compares three experimental configurations:

- TF-IDF features with Logistic Regression,
- transformer-based sentence embeddings with Logistic Regression,

- transformer-based sentence embeddings with linear SVM.

The research question of the project is:

Can transformer-based sentence embeddings outperform a TF-IDF baseline for Turkish fake news detection under the selected dataset, preprocessing pipeline, and classifier settings?

The contribution of this project is not the development of a new transformer architecture. Instead, it provides a controlled experimental comparison between a classical lexical baseline and transformer-based sentence representations. The results are analyzed with standard classification metrics and confusion matrices in order to understand which approach is more effective for the selected dataset.

## 2 Background and Related Work

Fake news detection has become an important research problem due to the rapid spread of misleading information through online news platforms and social media. In the literature, fake news detection is generally studied as a supervised text classification task, where a model learns to distinguish unreliable or fabricated news from reliable news using labeled examples. Shu et al. discuss fake news detection from a data mining perspective and emphasize that the problem is challenging because fake news is often designed to imitate legitimate news while spreading false or misleading claims [1]. Wang introduced the LIAR dataset as an early benchmark for automatic fake news detection, showing the importance of structured labeled datasets for evaluating classification models [2]. Perez-Rosas et al. also studied automatic fake news detection and highlighted that linguistic features can be useful for separating fake and legitimate news articles [3].

Traditional machine learning methods are still important baselines in text classification. A common approach is to represent documents with lexical features such as bag-of-words, n-grams, or TF-IDF, and then train classifiers such as Logistic Regression, Naive Bayes, or Support Vector Machines. These models are computationally efficient, easy to reproduce, and often effective when the dataset contains class-specific word patterns. In this project, TF-IDF is used as the baseline representation because it provides a simple and measurable way to evaluate how well fake and real news can be separated using word-level information.

Recent progress in natural language processing has been strongly influenced by transformer-based language models. The Transformer architecture introduced the self-attention mechanism, which allows a model to capture relationships between tokens without relying on recurrent sequence processing [4]. BERT later improved contextual language representation by pretraining deep bidirectional transformer models and adapting them to downstream NLP tasks [5]. Later studies such as RoBERTa showed that pretraining strategy, data size, and optimization choices can significantly affect transformer performance [6]. Efficient transformer variants such as DistilBERT also showed that smaller and faster models can be useful in practical NLP applications where computational resources are limited [7].

Multilingual transformer models are especially relevant for languages other than English. XLM-R demonstrated that cross-lingual representation learning can be scaled to many languages and can improve multilingual NLP performance [8]. This is important for Turkish fake news detection because Turkish has rich morphology, flexible word formation, and different linguistic patterns compared to English. Therefore, using a multilingual representation model can be a practical choice when working with Turkish text.

Sentence-level embedding models provide another practical way to use transformer-based representations. Sentence-BERT modifies pretrained BERT-style models using siamese and triplet network structures in order to produce semantically meaningful sentence embeddings [9]. These embeddings can be used as dense vector representations for classification, clustering, semantic search, and similarity-based tasks. In this project, the multilingual Sentence-Transformers model `paraphrase-multilingual-MiniLM-L12-v2` is used as a fixed feature extractor to generate sentence embeddings for Turkish news texts [10].

Turkish fake news detection is a more specific setting compared to widely used English fake news benchmarks. Previous Turkish-oriented studies show that language-specific resources and dataset characteristics are important for this task. Mertoglu and Genc studied lexicon generation for fake news detection and emphasized the role of lexical signals in misinformation-related text analysis [11]. Kulaksiz and Coskuncay studied fake news detection on mainstream media using natural language processing, which is closely related to the dataset used in this project [12]. The dataset used in this project is also publicly available on Hugging Face as a Turkish fake news detection dataset [13].

Based on this background, this project compares a classical TF-IDF baseline with transformer-based sentence embeddings. The purpose is not to claim that one family of models is universally superior, but to evaluate which representation performs better under the selected Turkish dataset, preprocessing pipeline, and classifier settings.

### 3 Dataset and Preprocessing

The dataset used in this project is the Turkish Fake News Detection dataset published on Hugging Face as `isakulaksiz/turkish-fake-news-detection`. The dataset contains Turkish news samples with binary labels indicating whether a news item is fake or real. This dataset was selected because it directly matches the project objective and provides labeled Turkish-language news data for supervised classification.

The preprocessing pipeline was implemented in `src/preprocessing.py`. The dataset was loaded using the Hugging Face `datasets` library and converted into a pandas `DataFrame`. In the original dataset structure, news content may include fields such as `title`, `description`, and label-related columns. For the final text representation, the `title` and `description` fields were combined when the `description` field was available. If the `description` field was not available, only the `title` field was used. This design allowed the model to use both headline-level and short-content information during classification.

The labels were normalized into two target classes: `fake` and `real`. When the dataset contained a `status` column, the value 1 was mapped to `real`, and the value 0 was mapped to `fake`. The preprocessing script also included additional label normalization logic for alternative column names such as `label` or `Resource`. This made the preprocessing pipeline more robust against small variations in dataset format.

After label normalization, rows with empty text values or missing labels were removed. The final processed dataset contained 5,326 samples. The processed data was saved into a clean two-column format with `text` and `label` fields. This simplified format was then used consistently by the TF-IDF baseline, embedding-based models, and evaluation scripts.

The dataset was split into training and test sets using an 80/20 stratified split. Stratification was used to preserve the fake/real class distribution in both subsets. A fixed `random_state=42` was used to make the split reproducible. As a result, the final training set contained 4,260 samples and the test set contained 1,066 samples.

The processed files were saved under the `data/processed/` directory as `train.csv` and `test.csv`. These files were used as the common input for all model training scripts. This ensured that all models were evaluated on the same test set and that the comparison between TF-IDF and transformer-based sentence embeddings remained fair.

## 4 Proposed Approach and Implementation

This project was designed as a comparative text classification study. The same preprocessed training and test sets were used for all models in order to make the comparison fair. The implementation includes one classical baseline model and two transformer-embedding-based models. The complete workflow consists of data preprocessing, feature extraction, model training, evaluation, and result aggregation.

### 4.1 Overall Pipeline

The experimental pipeline follows these steps:

1. Load the Turkish fake news dataset from Hugging Face.
2. Combine available textual fields into a single text column.
3. Normalize labels into two classes: fake and real.
4. Split the dataset into stratified training and test sets.
5. Train a TF-IDF + Logistic Regression baseline.
6. Generate transformer-based sentence embeddings.
7. Train Logistic Regression and SVM classifiers on the embeddings.
8. Evaluate all models using the same test set and classification metrics.

This pipeline was implemented using Python. The main libraries used in the project include `pandas` for data processing, `scikit-learn` for classical machine learning models and evaluation metrics, `datasets` for loading the Hugging Face dataset, `sentence-transformers` for generating sentence embeddings, `NumPy` for numerical arrays, `joblib` for saving model artifacts, and `matplotlib` for generating figures.

### 4.2 TF-IDF Baseline

The first model is a classical TF-IDF baseline. TF-IDF represents a document by assigning weights to terms based on their frequency in a document and their rarity across the corpus. In this project, the TF-IDF vectorizer was configured with a maximum of 20,000 features and an n-gram range of (1,2), meaning that both unigrams and bigrams were included.

After feature extraction, a Logistic Regression classifier was trained on the TF-IDF vectors. Logistic Regression was selected because it is a strong and commonly used baseline for binary text classification. It is also computationally efficient and easier to interpret compared to more complex models. In the implementation, the classifier used `random_state=42`, the `liblinear` solver, and `max_iter=1000`.

The TF-IDF model and vectorizer were saved as model artifacts. The script also produced a metrics CSV file and a confusion matrix figure for the test set.

### 4.3 Transformer-Based Sentence Embeddings

The second approach uses transformer-based sentence embeddings. Instead of representing text as sparse word-frequency vectors, each news text is converted into a dense vector representation. The embedding model used in this project is `paraphrase-multilingual-MiniLM-L12-v2`, which is available through the Sentence-Transformers framework.

This model was used as a fixed feature extractor. In other words, full transformer fine-tuning was not performed. Each input text was encoded into a 384-dimensional sentence embedding. These embeddings were then used as input features for classical machine learning classifiers.

Two classifiers were trained on the generated embeddings:

- Logistic Regression,
- linear Support Vector Machine.

The Logistic Regression model was trained with `random_state=42` and `max_iter=1000`. The SVM model used a linear kernel. This setup allowed the project to compare whether dense semantic representations improve performance over a lexical TF-IDF baseline.

### 4.4 Embedding Caching and Reproducibility

Computing transformer-based sentence embeddings can be more expensive than TF-IDF feature extraction. To reduce repeated computation, the project used a caching strategy. The train and test embeddings were saved as NumPy arrays after the first computation. When the embedding training script was executed again, the cached arrays were loaded instead of recomputing embeddings from the beginning.

The cached embedding files were stored as:

- `data/processed/embeddings_train.npy`,
- `data/processed/embeddings_test.npy`.

The trained classifiers and vectorizers were saved under the `models/` directory. Evaluation outputs, including metrics files, confusion matrices, and the final comparison table, were saved under the `results/` directory. This file organization supports reproducibility and makes it possible to rerun or inspect each stage of the experiment.

### 4.5 Project Modules

The implementation was organized into separate Python modules. The `preprocessing.py` file handles dataset loading, text construction, label normalization, and train/test splitting. The `train_tfidf.py`

file trains the TF-IDF baseline and saves its metrics, model, vectorizer, and confusion matrix. The `train_embeddings.py` file generates sentence embeddings, trains Logistic Regression and SVM classifiers, and saves their results. The `evaluate.py` file combines all metrics into a final comparison table and generates the F1-score comparison graph. Additional helper functions for reproducibility and metric saving were placed in `utils.py` and `embedding_utils.py`.

## 5 Evaluation and Results

The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. Since fake news detection has both false positive and false negative risks, F1-score is used as an important summary metric.

Table 1: Model Performance Comparison

| Model Type    | Classifier          | Accuracy      | Precision     | Recall        | F1-score      |
|---------------|---------------------|---------------|---------------|---------------|---------------|
| <b>TF-IDF</b> | Logistic Regression | <b>0.9381</b> | <b>0.9595</b> | 0.9054        | <b>0.9317</b> |
| Embedding     | Logistic Regression | 0.9043        | 0.8895        | <b>0.9074</b> | 0.8984        |
| Embedding     | SVM                 | 0.9109        | 0.9052        | 0.9034        | 0.9043        |

Table 1 shows that TF-IDF + Logistic Regression achieved the best result. It reached an accuracy of 0.9381 and an F1-score of 0.9317. Among the embedding-based models, SVM performed slightly better than Logistic Regression.

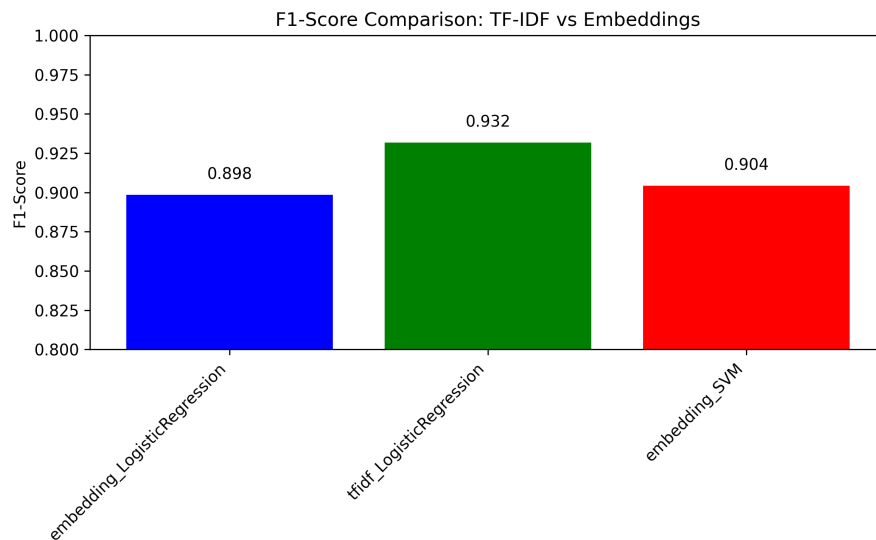


Figure 1: F1-score comparison of TF-IDF and embedding-based models

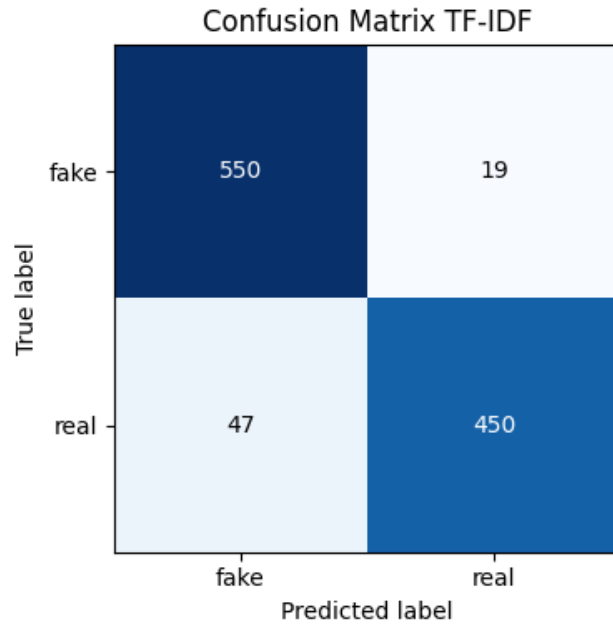


Figure 2: Confusion matrix of TF-IDF + Logistic Regression

The confusion matrix in Figure 2 shows that the TF-IDF model correctly classified 550 fake samples and 450 real samples. It produced 19 false positives and 47 false negatives.

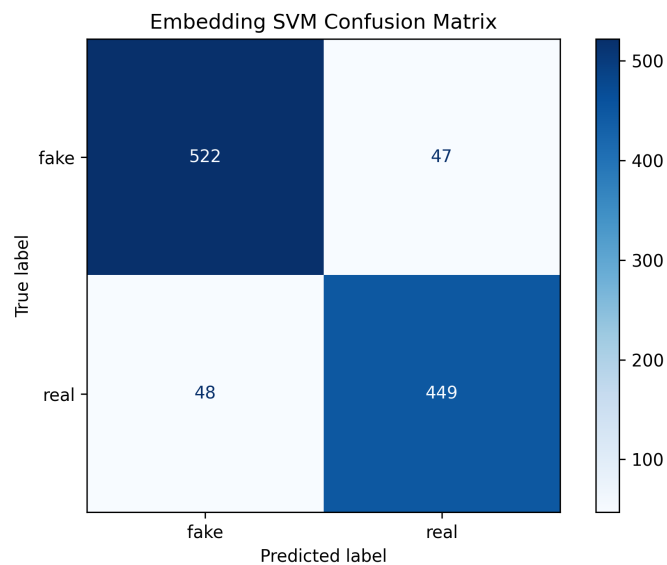


Figure 3: Confusion matrix of Embedding + SVM

The Embedding + SVM model correctly classified 522 fake samples and 449 real samples. It performed better than Embedding + Logistic Regression, but it was still below the TF-IDF baseline.

## 6 Discussion and Limitations

The results show that TF-IDF + Logistic Regression performed better than the transformer-based sentence embedding models in this experiment. This was not the expected result at the beginning, because transformer-based representations are usually stronger at capturing semantic meaning.

One possible reason is that this dataset may contain strong lexical patterns. Fake and real news samples may use different words, expressions, or headline styles. Since TF-IDF directly uses word-level information, it may capture these patterns better than the selected embedding model.

Another possible reason is that `paraphrase-multilingual-MiniLM-L12-v2` creates a fixed-size dense vector for each text. During this compression, some specific word-level signals may become less visible to the classifier.

This result should not be interpreted as transformer embeddings being generally worse than TF-IDF. The conclusion only applies to this dataset, preprocessing pipeline, embedding model, and classifiers. A fully fine-tuned Turkish transformer model could produce different results.

There are also some limitations. The project only uses text content. It does not use source credibility, publication date, author information, images, or social media sharing patterns. Also, some labels were normalized using heuristic rules, so minor label noise may exist.

## 7 Conclusion and Future Work

This project compared TF-IDF and transformer-based sentence embeddings for Turkish fake news detection. The best result was obtained with TF-IDF + Logistic Regression, which achieved an F1-score of 0.9317. The best embedding-based model was Embedding + SVM with an F1-score of 0.9043.

The results suggest that lexical features are very useful for this dataset. However, embedding-based models still produced competitive results and remain useful for representing semantic meaning.

Future work may include fine-tuning a Turkish transformer model such as BERTurk, testing larger datasets, comparing Turkish and English datasets, and combining TF-IDF features with dense embeddings. Metadata such as source information and publication dates could also be added to improve real-world fake news detection.

## References

- [1] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake news detection on social media: A data mining perspective,” *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017. [Online]. Available: <https://arxiv.org/abs/1708.01967>
- [2] W. Y. Wang, ““liar, liar pants on fire”: A new benchmark dataset for fake news detection,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2017, pp. 422–426. [Online]. Available: <https://arxiv.org/abs/1705.00648>
- [3] V. Pérez-Rosas, B. Kleinberg, A. Lefevre, and R. Mihalcea, “Automatic detection of fake news,” in *Proceedings of the 27th International Conference on Computational Linguistics*, 2018, pp. 3391–3401. [Online]. Available: <https://aclanthology.org/C18-1287/>
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [8] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 8440–8451. [Online]. Available: <https://arxiv.org/abs/1911.02116>
- [9] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [10] Sentence-Transformers, “paraphrase-multilingual-minilm-l12-v2,” Hugging Face Model Hub, Hugging Face, 2024. [Online]. Available: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- [11] U. Mertoğlu and B. Genç, “Lexicon generation for detecting fake news,” *arXiv preprint arXiv:2010.11089*, 2020. [Online]. Available: <https://arxiv.org/abs/2010.11089>
- [12] İ. Kulaksız and A. Coşkunçay, “Fake news detection on mainstream media using natural language processing,” *Black Sea Journal of Engineering and Science*, vol. 8, pp. 214–224, 2025. [Online]. Available: <https://dergipark.org.tr/en/pub/bsengineering/article/1527551>

- [13] İ. Kulaksız, “Tr-fakenews: Turkish fake news detection on mainstream media dataset,” Hugging Face Datasets, 2025. [Online]. Available: <https://huggingface.co/datasets/isakulaksiz/turkish-fake-news-detection>

## **AI Usage Statement**

AI tools were used for planning, code assistance, debugging, documentation support, and report drafting suggestions. All AI-assisted outputs were manually reviewed, edited, and verified by the project members. The final experiments, metrics, figures, and interpretations are based on the project code and saved result files. Detailed AI usage records are provided in `docs/ai-usage.md`.